

SameerTraderz Distribution System — SQA Manual

Testing Flow Guide

This guide is designed for the SQA team to manually test the API routes and verify that the database states, stock quantities, and financial ledgers update correctly after each transaction.

1. Independent Creations (Master Data)

These actions are straightforward. They only write to a single table in the database and have **no secondary effects** (no ledger entries, no stock changes, no transactions).

- **Create User** (`POST /api/auth/register`): Creates a user profile with hashed passwords.
 - **Create Category** (`POST /api/product-category`): Registers a product category.
 - **Create Product** (`POST /api/product`): Registers a product with `costPrice` , `sellingPrice` , and `lowStockLevel` . Initial stock quantity is set to 0.
 - **Create Customer** (`POST /api/customer`): Registers a customer. Outstanding `balance` starts at 0.00.
 - **Create Supplier** (`POST /api/supplier`): Registers a supplier. Outstanding `balance` starts at 0.00.
 - **Create Salesman** (`POST /api/salesman`): Registers a salesman (does not log in, acts as a tracking tag on sales).
 - **Create Expense Category** (`POST /api/expense-category`): Registers a category (e.g. Utility, Transport).
-

2. Stock Adjustments (Manual Inventory Correction)

Used when stock is manually adjusted (e.g., damaged products, expired stock, or physical count correction).

- **Trigger Route:** `POST /api/stock/adjust`
 - **Transactional Flow (Atomic):**
 1. Writes a `StockMovement` row with type `IN` (adds stock) or `OUT` (subtracts stock).
 2. Updates `Product.stockQuantity` by adding or subtracting the pieces directly.
 - **Verification Check:**
 - Check the `Product` table: is the `stockQuantity` updated correctly?
 - Check the `StockMovement` table: is there a matching log row?
 - *Constraint:* Try to subtract more stock than you have. The server must reject this with a `400 Bad Request` and roll back completely (stock must not become negative).
-

3. Purchases & Purchase Returns (Supplier Flow)

This flow handles buying stock from suppliers and returning damaged/wrong items back to them.

A. Recording a Purchase (`POST /api/purchase`)

- **What it does:** Increases your stock and records how much money you owe the supplier.
- **Transactional Flow (Atomic):**
 1. Creates a `Purchase` record (sequential doc number e.g. `PUR-000001`) and saves the line items in `PurchaseItem` .
 2. For every item, it increments `Product.stockQuantity` and logs a `StockMovement` row (`IN`).
 3. Inserts a credit line in `SupplierLedger` (indicating you owe them money).
 4. Updates `Supplier.balance` by adding the unpaid purchase amount (`total - paidAmount`).
 5. If `paidAmount > 0` , it logs a `SupplierPayment` row.
- **Verification Check:**
 - Product stock levels must increase.
 - Supplier balance must increase by the unpaid amount.
 - There must be corresponding `StockMovement` logs and a `SupplierLedger` entry.

B. Recording a Purchase Return (`POST /api/purchase-return`)

- **What it does:** Returns bought items to the supplier, decreasing your stock and reducing the amount you owe them.
- **Transactional Flow (Atomic):**
 1. Creates a `PurchaseReturn` record (doc number e.g. `PR-000001`) and saves `PurchaseReturnItem` lines.
 2. For every returned item, it decrements `Product.stockQuantity` and logs a `StockMovement` row (`OUT`).
 3. Inserts a debit line in `SupplierLedger` (reducing what you owe).
 4. Updates `Supplier.balance` by subtracting the return total.
- **Verification Check:**
 - Product stock levels must decrease.
 - Supplier balance must go down.

4. Sales Invoices & Sales Returns (Customer Billing Flow)

This flow handles selling items to customers (on cash or credit) and processing customer returns.

A. Recording a Sale/Invoice (`POST /api/invoice`)

- **What it does:** Sells items to a customer. Reduces your stock, records payment, and charges customer credit if unpaid.
- **Transactional Flow (Atomic):**
 1. Checks if the requested quantities are available. If any product is out of stock, the entire transaction is rejected and rolls back.
 2. Creates an `Invoice` record (doc number e.g. `INV-000001`) and saves `InvoiceItem` rows.
 3. **Cost Price Snapshotted:** Every `InvoiceItem` saves the product's current cost price in `costPriceAtSale` (this ensures reporting profit values stay accurate even if cost price is edited later).
 4. Decrements `Product.stockQuantity` and logs a `StockMovement` row (`OUT`) for each item.
 5. If the invoice has a credit component (`total - paidAmount > 0`), it writes a debit entry in `CustomerLedger` and increases `Customer.balance` .
 6. If `paidAmount > 0` , it logs a `CustomerPayment` row.
- **Verification Check:**
 - Product stock must decrease.
 - Customer balance must increase by the unpaid amount.
 - A `CustomerLedger` entry and `StockMovement` logs must be present.

B. Recording a Sales Return (`POST /api/sales-return`)

- **What it does:** Customer returns merchandise. Reverses stock and customer ledger balances.
- **Transactional Flow (Atomic):**
 1. Validates that the returned quantities do not exceed what was originally sold on that invoice.
 2. Creates a `SalesReturn` record (doc number e.g. `SR-000001`) and saves `SalesReturnItem` lines.
 3. Increments `Product.stockQuantity` and logs a `StockMovement` row (`IN`) for returned items.
 4. Logs a credit entry in `CustomerLedger` (reducing what they owe us) and updates `Customer.balance` .
- **Verification Check:**
 - Product stock levels must increase.
 - Customer balance must go down.

5. Payments & Settlements

This flow is used when a customer pays down their overall balance, or when we pay a supplier.

A. Recording a Customer Payment (`POST /api/payment/customer`)

- **What it does:** Customer pays off their debt.
- **Transactional Flow (Atomic):**
 1. Creates a `CustomerPayment` row.
 2. **Invoice Settlement:** If paying against a specific invoice, it updates the invoice's `paidAmount` and `balanceDue`, and sets `status` to `PAID` or `PARTIALLY_PAID`.
 3. Logs a credit entry in `CustomerLedger` and reduces `Customer.balance`.
 4. *Constraint:* Try to pay more than what is due on the invoice or overall customer balance. The server must reject this with a `400 Bad Request`.

B. Recording a Supplier Payment (`POST /api/payment/supplier`)

- **What it does:** We pay the supplier.
- **Transactional Flow (Atomic):**
 1. Creates a `SupplierPayment` row.
 2. Logs a debit entry in `SupplierLedger` and reduces `Supplier.balance`.
 3. *Constraint:* Try to pay more than the overall supplier balance. The server must reject this.

6. Targets & Expenses

- **Set Salesman Target** (`POST /api/sales-target/salesman/:id/target`): Creates or updates target amounts for a salesman for a month (e.g. `"2026-07"`).
- **Check Achievement** (`GET /api/sales-target/salesman/:id/achievement?month=2026-07`): Sums the salesman's gross sales in that calendar month, subtracts returns, compares it against the target, and calculates the percentage achieved.
- **Log Expense** (`POST /api/expense`): Logs a business expense. Writes to `Expense` table with no inventory side effects.

7. Reports & Aggregations

- **Dashboard** (`GET /api/report/dashboard`): Returns running totals. Check that they match your manual database row counts (Receivables, Payables, Monthly Expenses, Low Stock alerts, and Top 5 Selling Products).
- **Receivables aging** (`GET /api/report/customer-ledger`): Check that customers with outstanding balances have their unpaid invoices grouped correctly into `0-30 days`, `31-60 days`, or `60+ days` buckets based on `invoiceDate`.